

Chapter 1: Angular Architecture

1. Overview

Angular is a platform and application design framework for building single-page applications (SPAs) using HTML, TypeScript, and a component-driven architecture. Unlike a simple UI library, Angular ships as a full, opinionated ecosystem: a component/template compiler, a dependency injection (DI) system, a reactive change-detection runtime, a router, a forms module, an HTTP client, and a CLI/build toolchain (esbuild/Vite-based since Angular 17+, Ivy compiler since Angular 9+).

The reason Angular exists as a "batteries-included" framework rather than a thin rendering library is that large, long-lived enterprise applications need consistent answers to problems that recur in every app: how do components talk to each other, how is state kept in sync with the DOM, how is code organized and lazy-loaded, how is testability guaranteed, how are cross-cutting concerns (logging, auth, caching) injected without tight coupling. Angular's architecture answers all of these with a fixed set of building blocks and a compiler that understands them at compile time — which is what enables tree-shaking, dead-code elimination, and ahead-of-time (AOT) compilation.

At its core, an Angular application is a tree of **components**, each backed by a **class** (the "controller") and a **template** (HTML with Angular-specific syntax). Components are composed into a tree. Cross-cutting logic is extracted into **services** and handed to components via **dependency injection**. The whole tree is bootstrapped into the DOM by the **Angular runtime**, which also drives **change detection** — the mechanism that keeps the view synchronized with the component state.

Since Angular 14–19, the framework has moved from a **mandatory NgModule-based architecture** to a **standalone-first architecture**, where components, directives, and pipes declare their own dependencies directly and NgModules become optional. As of Angular 17+, standalone is the default generated by the CLI, and Angular 19 marks NgModules as being in maintenance mode for new code. Understanding both models — and how the compiler/runtime treat them as fundamentally similar under the hood — is essential for interviews, since most production codebases are still transitioning between the two.

2. Core Concepts

2.1 The Building Blocks

Angular's public architecture is made of a small number of primitives:

- **Components** — `@Component` decorated classes; the fundamental UI building block. Combines a template (view) with a class (state + behavior) and metadata (selector, styles, change detection strategy).
- **Directives** — `@Directive` decorated classes that add behavior to existing DOM elements (attribute directives like `NgClass`, `NgStyle`) or that change the DOM structure (structural directives like `*ngIf`, `*ngFor`, or the newer built-in control flow `@if` / `@for`).
- **Pipes** — `@Pipe` decorated classes that transform data for display in templates (`date`, `currency`, custom pipes). Pure by default (memoized on reference-equality of inputs).
- **Services and Dependency Injection** — plain classes (usually `@Injectable`) that encapsulate reusable logic/state, provided through Angular's hierarchical DI container and injected into components/directives/other services via constructor injection or the `inject()` function.
- **Modules (NgModule) / Standalone components** — the organizational unit that used to be mandatory

(`@NgModule`) and is now optional; the mechanism by which the compiler learns which components/directives/pipes are visible to which templates.

- **Router** — maps URL paths to components (or lazy-loaded chunks), managing navigation, guards, resolvers.
- **Template syntax & binding engine** — interpolation `{{ }}`, property binding `[prop]`, event binding `(event)`, two-way binding `[(ngModel)]`, structural directives, and (Angular 17+) the built-in control-flow syntax `@if`, `@for`, `@switch`, `@defer`.

2.2 Compiler vs. Runtime — the Two Halves of Angular

This is the single most important architectural distinction for a senior interview.

The Compiler (Ivy, `@angular/compiler-cli`)

Angular ships a compiler that transforms `@Component` / `@Directive` / `@Pipe` / `@NgModule` decorated classes and their HTML templates into plain TypeScript/JavaScript instruction sets *before* the app ships to the browser (AOT — Ahead-Of-Time compilation is the default and recommended mode; JIT — Just-In-Time compilation in the browser is legacy and effectively removed as a first-class path since v9+).

The compiler's job:

- Parses the HTML template into an AST.
- Type-checks bindings against the component class (`strictTemplates`), catching typos and type mismatches at build time.
- Emits a **component definition object** (`ɵcmp`) containing a `ɵtemplate` function made of low-level **instructions** (`ɵɵelementStart`, `ɵɵproperty`, `ɵɵlistener`, `ɵɵtext`, etc.) that directly build/update the DOM — no runtime HTML parsing.
- Performs **tree-shaking-friendly** codegen: unused directives/pipes/components are never referenced, so bundlers can drop them.

The Runtime (`@angular/core`)

The runtime is the library loaded in the browser that:

- Executes the compiled instruction functions to create and update DOM nodes.
- Owns the **change detection** engine that decides *when* to re-run those update instructions.
- Owns the **dependency injection container(s)**.
- Owns component/directive **lifecycle** management.
- Owns **zone.js integration** (optional since zoneless change detection, Angular 18+ experimental / 19+ more stable) that decides *when* to trigger a change-detection pass.

The key insight: with Ivy, compiled components are **self-contained and locality-based** — a component's compiled output depends only on itself and its own `imports` / declaring module, not on the whole application's compilation graph. This is what allows AOT compilation to be incremental and fast, and what makes standalone components possible at all (no need for a global "module graph" to resolve template dependencies).

2.3 Ivy Instructions & the Component Definition

Every compiled component ends up with a static field, e.g.:

```

static cmp = defineComponent({
  type: MyComponent,
  selectors: ["app-my"],
  standalone: true,
  decls: 2,
  vars: 1,
  template: function MyComponent_Template(rf, ctx) {
    if (rf & 1) { // creation mode
      elementStart(0, "div");
      text(1);
      elementEnd();
    }
    if (rf & 2) { // update mode
      advance(1);
      textInterpolate(ctx.value);
    }
  }
});

```

The `rf` (render flags) parameter is bitwise-split into **creation mode** (build DOM once) and **update mode** (re-bind values on every change-detection pass). This dual-mode template function is the core internal mechanic of Ivy rendering — it's why Angular templates are cheap to re-run: creation-mode instructions execute once; only update-mode bindings execute on every CD pass, and they are simple property/attribute writes, not template re-parsing.

2.4 Standalone Architecture vs. NgModule Architecture

NgModule model (classic, pre-v14, still valid):

- `@NgModule({ declarations, imports, exports, providers, bootstrap })` groups components/directives/pipes and declares what's visible to what.
- A component must be declared in exactly one module.
- To use a directive/component/pipe in a template, its module must import the module that exports it.
- `AppModule` with `bootstrapModule(AppModule)` is the app entry point; `BrowserModule` provides browser-specific renderer/DOM services.
- Lazy loading works via `LoadChildren` returning a module, loaded through the router.

Standalone model (default since Angular 17, stable since 14):

- A component declares `standalone: true` (implicit/default in v19+, no need to write it) and lists its own template dependencies directly in an `imports: [...]` array on the `@Component` decorator.
- No `NgModule` is required at all — not even for the root component.
- Bootstrapping uses `bootstrapApplication(AppComponent, appConfig)` instead of `bootstrapModule`.
- App-wide providers (router, HTTP client, etc.) are configured via **providers functions**: `provideRouter()`, `provideHttpClient()`, `provideAnimations()`, etc., passed into `bootstrapApplication`'s `appConfig`, instead of being module imports.
- Lazy loading works via `LoadComponent` (returns a standalone component directly) or `LoadChildren` (returns routes), no module wrapper needed.

- `ɵcmp.dependencies` on the compiled definition (derived from the component's own `imports` array) replaces the module's `declarations/imports` resolution — this is what makes each component's compilation "local."

Under the hood, both models compile to the **same** `ɵɵdefineComponent` shape; a component's "module" is really just a bag that used to be needed to resolve *which directives/pipes are visible in this template*. Standalone components resolve that visibility locally via their own `imports` array, eliminating an entire category of "forgot to add to declarations/imports" errors and enabling finer-grained tree-shaking and simpler lazy-loading boundaries.

2.5 Dependency Injection Architecture

Angular DI is **hierarchical**: injectors form a tree that mirrors (roughly) the component tree, plus a module injector hierarchy for NgModule apps.

- **Root injector** / `providedIn: 'root'`: a single application-wide singleton injector; services registered this way are tree-shakable — if nothing injects the service, the compiler can drop it since the provider is defined as static metadata on the injectable itself rather than in a module's `providers` array.
- **Platform injector**: sits above the root injector; shared across multiple Angular applications on the same page (rare, but relevant for micro-frontends).
- **Environment injectors**: the injector type introduced by Ivy (`EnvironmentInjector`) that backs both `providedIn: 'root'` and `providedIn: <SomeStandaloneComponent>` styles, and module-level `providers`; created lazily per lazy-loaded route boundary too.
- **Element injectors**: per-component/per-directive injectors created from the `providers/viewProviders` arrays on `@Component/@Directive`; these follow the DOM tree and support `@Optional`, `@Self`, `@SkipSelf`, `@Host` resolution modifiers.
- **Resolution algorithm**: when a token is requested, Angular walks up the element injector tree first (respecting `@Host/@SkipSelf`), then falls back to the environment injector tree (module/root), then platform injector, until it finds a provider or throws `NullInjectorError`.

Modern DI is typically consumed via the `inject()` function (works in field initializers, factory functions, and constructors) rather than only constructor parameter injection, which was the only option pre-v14.

2.6 Change Detection Architecture

Angular's default change detection runs **top-down, once per component tree per CD cycle**, checking every component's bindings in a deterministic order matching the component tree.

- **Zone.js**: patches async browser APIs (`setTimeout`, `addEventListener`, `Promise`, XHR, etc.) so that after *any* such async task completes, Angular knows "something might have changed" and triggers `ApplicationRef.tick()`, which walks the entire component tree checking bindings.
- `ChangeDetectionStrategy.Default`: every component is checked on every tick, regardless of whether its inputs changed.
- `ChangeDetectionStrategy.OnPush`: a component (and its subtree) is only checked when: (a) an `@Input()` reference changes, (b) an event originates from within the component (or a descendant) itself, (c) an observable bound via the `async` pipe emits, or (d) change detection is manually triggered (`ChangeDetectorRef.markForCheck()` or `detectChanges()`). OnPush is the standard senior-level performance lever.

- **Signals (Angular 16+) and zoneless (Angular 18+/19+)**: signals are a fine-grained reactive primitive (`signal()`, `computed()`, `effect()`) that track *exactly* which consumers read them, so updating a signal can trigger updates only for the components/templates that actually read it — no `zone.js` and no whole-tree walk required. Angular is moving toward **zoneless change detection** (`provideExperimentalZonelessChangeDetection()` / stabilizing across 18–20) where signals (or explicit `markForCheck`) are the sole trigger for re-render, removing `zone.js` from the bundle entirely for better performance and simpler stack traces.

2.7 Bootstrapping Process (Mental Model End-to-End)

Standalone bootstrap (modern default):

1. `main.ts` calls `bootstrapApplication(AppComponent, appConfig)`.
2. Angular creates the **platform** (`platformBrowser()` internally) — the outermost injector, shared if multiple apps exist on the page.
3. Angular creates the **root environment injector**, registering the providers from `appConfig.providers` (e.g., `provideRouter`, `provideHttpClient`, `provideZoneChangeDetection`).
4. Angular instantiates `AppComponent`, running its constructor/field initializers (DI resolution happens here), and creates its element injector and view.
5. Angular inserts the component's root DOM node in place of the `<app-root>` host element found in `index.html`.
6. First change detection pass ("creation mode" render) executes each template's creation-mode instructions, building the full DOM.
7. `ApplicationRef` marks the app as **stable** once initial rendering + any pending async initializers (`APP_INITIALIZER` / `provideAppInitializer`) resolve — this is what `whenStable()` /testing utilities and Angular Universal (SSR) hydration wait on.
8. From here, the runtime is "live": `zone.js` (or signals) triggers subsequent change-detection ticks in response to events/async work.

NgModule bootstrap (classic) differs only in steps 1–3:

`platformBrowserDynamic().bootstrapModule(AppModule)` compiles/reads `AppModule`, resolves its `imports` / `providers` / `declarations` graph, and instantiates whichever component is listed in `AppModule`'s `bootstrap: [AppComponent]` array. Everything from DOM insertion onward is identical, because both paths converge on the same Ivy component definitions and the same `ApplicationRef`.

2.8 The Angular Compiler Pipeline (Build-Time View)

1. **TypeScript compilation** (`tsc` / `esbuild`) type-checks and transpiles `.ts` files.
2. **Angular template compiler** (`@angular/compiler-cli`, invoked via the Angular CLI/ `ngc`) parses `.html` templates (or inline templates), applies `strictTemplates` type-checking against the component class, and emits the `@ɵdefineComponent` instruction functions, merging them onto the component class output.
3. **Metadata extraction** determines each component's `selector`, `inputs` / `outputs` (from `@Input()` / `@Output()` or the signal-based `input()` / `output()` APIs), `standalone` flag, and `dependencies` (its resolved directive/pipe/component matches).

4. **Bundling** (esbuild + Vite dev server since Angular 17's new build system, replacing webpack as default): tree-shakes unused providers/components/pipes, applies AOT-friendly dead code elimination (Angular's compiler emits code shaped for `ngDevMode` guards to strip dev-only warnings/checks in production builds).
5. **Optional partial compilation** for published libraries (Ivy "partial" mode + the Angular Package Format, APF) so libraries ship compiler-agnostic intermediate code that gets fully linked by the consuming app's own Angular compiler version — this decouples library and app compiler versions.

3. Code Examples

3.1 Standalone Component (modern default)

```
// user-card.component.ts
import { Component, input, output } from '@angular/core';
import { CommonModule } from '@angular/common';

@Component({
  selector: 'app-user-card',
  standalone: true, // implicit default in v19+, shown here for clarity
  imports: [CommonModule],
  template: `
    <div class="card">
      <h3>{{ name() }}</h3>
      @if (isAdmin()) {
        <span class="badge">Admin</span>
      }
      <button (click)="selected.emit(name())">Select</button>
    </div>
  `,
})
export class UserCardComponent {
  name = input.required<string>();
  isAdmin = input(false);
  selected = output<string>();
}
```

3.2 Bootstrapping a Standalone App

```
// main.ts
import { bootstrapApplication } from '@angular/platform-browser';
import { provideRouter } from '@angular/router';
import { provideHttpClient, withInterceptors } from '@angular/common/http';
import { provideZoneChangeDetection } from '@angular/core';
import { AppComponent } from './app/app.component';
import { routes } from './app/app.routes';
import { authInterceptor } from './app/auth.interceptor';

bootstrapApplication(AppComponent, {
  providers: [
    provideZoneChangeDetection({ eventCoalescing: true }),
  ],
})
```

```

    provideRouter(routes),
    provideHttpClient(withInterceptors([authInterceptor])),
  ],
}).catch((err) => console.error(err));

```

3.3 Classic NgModule Bootstrap (for comparison / legacy codebases)

```

// app.module.ts
import { NgModule } from '@angular/core';
import { BrowserModule } from '@angular/platform-browser';
import { AppComponent } from './app.component';
import { UserCardModule } from './user-card/user-card.module';

@NgModule({
  declarations: [AppComponent],
  imports: [BrowserModule, UserCardModule],
  providers: [],
  bootstrap: [AppComponent],
})
export class AppModule {}

// main.ts
import { platformBrowserDynamic } from '@angular/platform-browser-dynamic';
import { AppModule } from './app/app.module';

platformBrowserDynamic().bootstrapModule(AppModule)
  .catch((err) => console.error(err));

```

3.4 Dependency Injection Across the Hierarchy

```

import { Injectable, inject, InjectionToken } from '@angular/core';

export const API_BASE_URL = new InjectionToken<string>('API_BASE_URL');

@Injectable({ providedIn: 'root' }) // tree-shakable singleton, resolved at the root env
injector
export class UserService {
  private http = inject(HttpClient); // inject() usable in field initializers
  private baseUrl = inject(API_BASE_URL); // custom token resolved wherever it's provided

  getUser(id: string) {
    return this.http.get(`${this.baseUrl}/users/${id}`);
  }
}

// Providing at the component level creates a NEW instance per component subtree,
// shadowing the root-provided singleton for that branch of the tree.
@Component({
  selector: 'app-feature',
  standalone: true,
  providers: [UserService], // element-injector-scoped, separate instance from root
})

```

```

    template: `...`,
  })
  export class FeatureComponent {}

```

3.5 OnPush + Signals (modern reactive component)

```

import { Component, ChangeDetectionStrategy, signal, computed, effect } from
  '@angular/core';

@Component({
  selector: 'app-counter',
  standalone: true,
  changeDetection: ChangeDetectionStrategy.OnPush,
  template: `
    <p>Count: {{ count() }}</p>
    <p>Doubled: {{ doubled() }}</p>
    <button (click)="increment()"></button>
  `,
})
export class CounterComponent {
  count = signal(0);
  doubled = computed(() => this.count() * 2); // recomputed only when `count` changes

  constructor() {
    effect(() => console.log('count changed to', this.count()));
  }

  increment() {
    this.count.update((c) => c + 1); // triggers fine-grained update, no zone.js required
  }
}

```

3.6 Lazy Loading with Standalone Routes

```

// app.routes.ts
import { Routes } from '@angular/router';

export const routes: Routes = [
  {
    path: 'dashboard',
    loadComponent: () =>
      import('./dashboard/dashboard.component').then((m) => m.DashboardComponent),
  },
  {
    path: 'admin',
    loadChildren: () => import('./admin/admin.routes').then((m) => m.ADMIN_ROUTES),
  },
];

```


4. Internal Working

Template compilation to instructions. The Angular compiler does not ship an HTML parser to the browser. Every template is compiled once, at build time, into a JavaScript function containing calls like `ɵɵelementStart`, `ɵɵproperty`, `ɵɵlistener`, `ɵɵtemplate` (for structural content/embedded views), and `ɵɵadvance` (moves an internal cursor rather than re-looking-up nodes). This is why Angular templates are fast: the "diffing" isn't a virtual-DOM diff against a tree — it's a flat, ordered sequence of primitive instructions re-executed in the same order every time, writing directly to already-created real DOM nodes.

Locality principle (Ivy). Pre-Ivy (View Engine), compiling `ComponentA`'s template required knowledge of the entire `NgModule` graph, because directive matching happened globally at module-compile time. Ivy made each component's compilation depend only on its own metadata + a locally resolvable list of directives (`imports` for standalone, or resolved via the enclosing module for declared components). This locality is precisely what makes standalone components not just a syntactic convenience but an architectural simplification: compilation units shrink, incremental rebuilds get faster, and dead code elimination becomes trivial per-component.

Change detection tree walk. `ApplicationRef.tick()` invokes `detectChanges` starting from each root component, and Angular walks the **logical view tree** (not the raw DOM tree) depth-first. For `Default` strategy components, every binding's update instruction re-executes and is compared against the last-seen value (this is why bindings must be pure/cheap — they run on every tick). For `OnPush` components, Angular sets an internal `dirty / CheckAlways` flag; the subtree is skipped entirely unless one of the four triggers (input ref change, local event, async pipe emission, explicit `markForCheck`) flips that flag.

Zone.js monkey-patching. Zone.js overrides global async APIs so it can wrap every callback in a "zone" that notifies Angular on exit. This is why change detection can appear to run "automatically" even though nothing in your component code calls it — it's zone.js's patched `setTimeout` / event listeners invoking `NgZone.onMicrotaskEmpty`, which schedules `tick()`. This is also precisely why change detection historically ran on unrelated events anywhere in the app (any zone-patched async callback firing triggers a full tree check), and why `OnPush`/signals/zoneless exist — to opt out of this coarse global trigger.

DI resolution algorithm. When a class requests a token, Angular first looks at the **element injector** chain (the DOM-tree-shaped hierarchy built from each component/directive's own `providers`), walking upward, honoring `@Self` (only this injector), `@SkipSelf` (skip this injector, start at parent), and `@Host` (stop at the host boundary — typically the component that owns the current view). If unresolved, it falls through to the **module/environment injector** hierarchy (feature module → root → platform), where `providedIn: 'root'` services live. If still unresolved and no `@Optional` was specified, Angular throws `NullInjectorError: No provider for X`.

Tree-shakable providers. `@Injectable({ providedIn: 'root' })` doesn't register the provider in any array a bundler must statically include — it attaches a factory function directly to the injectable class (`ɵɵprov`). If no code path ever injects the service, nothing references `ɵɵprov`, and the bundler's tree-shaking safely drops the entire class. This is structurally impossible with services listed in an `NgModule`'s `providers: []` array, since the module object itself holds a live reference regardless of usage — one of the concrete technical wins standalone/tree-shakable DI has over classic `NgModule` providers.

Hydration & SSR (Angular 16+ non-destructive hydration). On the server, Angular renders the component tree to static HTML. On the client, instead of tearing down and re-rendering the DOM (as older Angular Universal versions did, causing flicker), Angular's hydration reuses the existing DOM nodes and re-attaches event listeners/bindings by matching against serialized state markers embedded in the HTML — reading the same compiled instruction functions in a "claim existing node" mode rather than a "create new node" mode.

5. Edge Cases & Gotchas

- **Mixing standalone and NgModule code:** a standalone component can still be declared into (imported by) an `NgModule` via that module's `imports` array (not `declarations` — standalone components/directives/pipes are *imported*, never declared). A classic (non-standalone) component, however, cannot be added to a standalone component's `imports` array — it must remain governed by its `NgModule`; you import that module instead.
- **`providedIn: 'root'` does not always mean one instance app-wide:** if a lazy-loaded module (classic architecture) also lists the same service in its own `providers` array, or if a component provides it locally, a *new* instance is created for that injector subtree, shadowing the root singleton silently — a classic "why is my service state not shared" bug.
- **OnPush + mutating objects in place:** if an `@Input()` object reference doesn't change (e.g., you `push()` into an existing array instead of reassigning), an OnPush component will **not** re-render, because Angular checks input identity, not deep equality. This trips up almost every developer at least once.
- **Async pipe subscription timing:** the `async` pipe automatically marks an OnPush component dirty on emission and unsubscribes on destroy — but only for the specific component whose template holds the pipe; nested components below it still need their own trigger (input change, event, or their own `async` pipe) to re-render.
- **Zone.js and third-party libraries:** libraries that schedule callbacks outside of any patched API (e.g., some WebSocket libraries, `requestIdleCallback` in older Zone.js versions) don't trigger change detection automatically — a common "my data updated but the screen didn't" bug, solved via `NgZone.run()` or manual `markForCheck()`.
- **`effect()` runs during change detection, not synchronously on signal write:** assuming an effect fires immediately/synchronously like a `computed()` read can produce confusing timing bugs and infinite-loop guards if the effect itself writes to a signal it also reads.
- **Standalone bootstrapping still needs `provideRouter` / `provideHttpClient` etc.:** forgetting these provider functions in `appConfig.providers` is the standalone-era equivalent of forgetting `RouterModule.forRoot()` / `HttpClientModule` import — same failure mode (`NullInjectorError` for `Router` / `HttpClient`), different syntax.
- **`forRoot()` / `forChild()` pattern:** classic modules like `RouterModule.forRoot(routes)` vs `RouterModule.forChild(routes)` exist specifically to avoid re-providing router-wide singletons (like `Router`, `Location`) every time a feature module imports `RouterModule` — a pattern interviewers often ask about because it demonstrates understanding of the module/provider distinction.
- **AOT-only bindings:** AOT's `strictTemplates` will catch type errors in bindings (e.g., passing a `string` where a component expects a `number @Input()`) that JIT mode would silently allow at runtime — teams sometimes get surprised when enabling strict mode on an old codebase surfaces dozens of previously-hidden template bugs.
- **Circular DI:** two services injecting each other via constructor injection throws a circular dependency error at instantiation; resolved via `Injector.get()` deferred lookup, restructuring responsibilities, or (rarely appropriate) `forwardRef`.
- **View encapsulation default (`Emulated`):** Angular's default `ViewEncapsulation.Emulated` scopes CSS by rewriting selectors with attribute hashes (`_ngcontent-xyz`) — this is not real Shadow DOM isolation;

global styles can still leak in, and `::ng-deep` (deprecated, still widely used) punches through it, which is a frequent architecture-hygiene topic in interviews.

6. Interview Questions & Answers

Q1. What are the fundamental building blocks of an Angular application?

A: Components (view + logic), directives (attribute/structural behavior), pipes (display-time data transforms), services (reusable logic/state), and the DI system that wires services into consumers. Organizationally, either `NgModules` (classic) or standalone components/directives/pipes (modern) determine what's visible to what. The router, HTTP client, and forms are first-party modules/APIs built on top of these same primitives.

Q2. What is the difference between the Angular compiler and the Angular runtime?

A: The compiler (`@angular/compiler-cli`, Ivy) runs at build time (AOT), turning `@Component` templates into JavaScript instruction functions (`ɵɵdefineComponent` with creation/update mode instructions) and performing template type-checking. The runtime (`@angular/core`, shipped to the browser) executes those instruction functions, owns change detection, dependency injection containers, and component lifecycle. The compiler produces static code; the runtime executes and reacts to it.

Interviewer intent: This checks whether the candidate understands Angular isn't "just a runtime library" but has a build-time code-generation step that fundamentally shapes performance (tree-shaking, no runtime template parsing) — a common line separating mid-level from senior understanding.

Q3. Walk through what happens when an Angular standalone app bootstraps.

A: `bootstrapApplication(AppComponent, appConfig)` creates a platform injector, then a root environment injector populated from `appConfig.providers` (router, HTTP client, zone config, etc.), instantiates `AppComponent` (running its constructor/injections), inserts its root DOM node into `<app-root>`, runs the first ("creation mode") change-detection pass to build the initial DOM, and marks the app "stable" once pending async initializers resolve. Subsequent renders are driven by change-detection ticks triggered by zone.js or signals.

Q4. Why did Angular introduce standalone components, and what problem in the NgModule model did they solve?

A: `NgModules` required every component/directive/pipe to be declared in exactly one module and required consuming modules to import the declaring module just to use a template dependency — a lot of ceremony, frequent "forgot to add to declarations" errors, and compilation units that depended on the whole module graph (View Engine) rather than being locally resolvable. Standalone components let each component list its own template dependencies via `imports`, removing `NgModules` as a mandatory concept, simplifying lazy-loading (`LoadComponent` vs. wrapping every lazy feature in its own module), and enabling finer-grained, per-component tree-shaking.

Interviewer intent: Probing whether the candidate can articulate an actual architectural motivation rather than reciting "standalone is the new syntax" — shows whether they understand *why* a major framework migration happened.

Q5. Explain Angular's dependency injection hierarchy.

A: DI is tree-shaped, mirroring roughly the component/module structure. Element injectors are created per-component/directive from their own `providers`/`viewProviders` and follow the DOM tree; environment injectors back `providedIn: 'root'` and module-level providers, plus lazy route boundaries; a platform injector sits above everything, shared across possibly multiple Angular apps on one page. Resolution walks the element injector chain first (respecting `@Self`/`@SkipSelf`/`@Host`), then the environment injector chain, throwing `NullInjectorError` if nothing is found (unless `@Optional`).

Q6. What's the difference between `providedIn: 'root'` and adding a service to a component's `providers` array?

A: `providedIn: 'root'` registers a singleton at the root environment injector, tree-shakable if unused, shared app-wide. Adding the same service to a component's `providers` array creates a new element-injector-scoped instance for that component's subtree, shadowing the root instance for any injection happening within that subtree — useful for per-feature/per-instance state (e.g., a form-wizard service that must reset per usage) but a common source of "why isn't my state shared" bugs when done accidentally.

Q7. How does Angular's default change detection actually get triggered without you calling anything?

A: Zone.js monkey-patches global async APIs (`setTimeout`, DOM event listeners, `Promise`, XHR, etc.).

Whenever a patched async callback finishes running, zone.js notifies `NgZone`, which schedules `ApplicationRef.tick()`. `tick()` walks the entire logical component tree, depth-first, re-executing each component's update-mode template instructions and comparing binding values used to decide whether corresponding DOM writes are needed.

Interviewer intent: Tests whether the candidate actually understands zone.js's role rather than treating change detection as "magic" — a prerequisite for reasoning about performance and about zoneless Angular.

Q8. What is `OnPush` change detection and when does an `OnPush` component re-render?

A: `ChangeDetectionStrategy.OnPush` opts a component (and, effectively, its subtree unless children have their own triggers) out of being checked on every global tick. It's only checked when: an `@Input()` reference changes, an event handler originates inside the component or a descendant, an `async`-pipable observable/signal emits, or `ChangeDetectorRef.markForCheck()` / `detectChanges()` is called explicitly. This requires treating inputs as immutable — mutating an object/array in place won't trigger a re-check because Angular compares references, not deep equality.

Q9. How do Angular Signals change the change-detection story compared to zone.js?

A: Signals are a fine-grained reactive primitive: `signal()` holds a value and tracks its readers; `computed()` derives a memoized value recomputed only when a dependency signal changes; `effect()` reruns a side-effecting function when its signal dependencies change. Because Angular's runtime can know exactly which template bindings read which signals, updating a signal can, in principle, re-render only the affected bindings/components rather than triggering a global zone.js-driven tree walk — enabling zoneless change detection, which removes zone.js from the bundle and relies on signals (plus explicit host/marking hooks) as the sole re-render trigger.

Q10. What is the "locality principle" in Ivy, and why does it matter architecturally?

A: Under View Engine (pre-Ivy), compiling a component's template required resolving the entire NgModule graph to know which directives/pipes matched selectors in that template, coupling compilation of any one component to the whole app's module graph. Ivy made each component's compiled output depend only on its own metadata and a locally resolvable set of dependencies (its own `imports` for standalone, or its declaring module's resolved exports otherwise). This is what makes incremental/AOT compilation faster, makes standalone components technically possible, and improves tree-shaking, since a component unambiguously references only what it actually uses.

Interviewer intent: This is a compiler-internals question aimed squarely at distinguishing candidates who've only used Angular from those who understand why Ivy was a rewrite, not just a renderer swap.

Q11. Why are tree-shakable providers (`providedIn: 'root'`) more bundle-efficient than NgModule `providers` arrays?

A: `providedIn: 'root'` attaches the provider factory as static metadata directly on the injectable class (`@prov`). If nothing in the compiled app ever injects that service, no code references `@prov`, so the bundler can

safely tree-shake the entire class and its dependencies away. A service listed in an `NgModule`'s `providers: []` array is referenced by the (retained) module object regardless of whether anything injects it, so it cannot be safely removed — the module itself is a live root the bundler must keep.

Q12. Explain the difference between AOT and JIT compilation, and why AOT is the default/required mode today.

A: JIT (Just-In-Time) compiles templates in the browser at application startup, requiring the compiler itself to ship as part of the bundle and delaying first render. AOT (Ahead-Of-Time) compiles templates during the build, producing plain instruction-function JavaScript with no compiler needed in the browser — smaller bundles, faster startup, and build-time template type-checking (`strictTemplates`) that catches binding errors before deployment. Since Angular 9 (Ivy default) and especially post-v13 (JIT compiler support trimmed further), AOT is effectively the only supported production path; JIT survives mainly for certain dynamic-compilation edge cases and some testing scenarios.

Q13. What's the difference between `@Self`, `@SkipSelf`, `@Host`, and `@Optional` in DI?

A: These are injection modifiers controlling where in the injector hierarchy resolution starts/stops and whether failure is tolerated. `@Self` restricts resolution to the requesting element's own injector only (no walking up). `@SkipSelf` starts resolution at the parent injector, skipping the local one — commonly used to fetch an ancestor instance of the same token (e.g., a parent form's `FormControl`). `@Host` stops resolution at the host component boundary of the current view, useful in directive/component authoring to avoid reaching into an arbitrary ancestor. `@Optional` suppresses the `NullInjectorError` and resolves to `null` if no provider is found, instead of throwing.

Q14. In a large enterprise app, would you choose OnPush everywhere, standalone components, and lazy loading by default? Justify the trade-offs.

A: Generally yes for a modern app: OnPush drastically reduces unnecessary change-detection work in large trees (at the cost of requiring immutable data discipline and awareness of the four re-check triggers); standalone components reduce boilerplate/coupling and align with where Angular's tooling/ecosystem is heading (CLI schematics, official docs, and library authors default to it); lazy loading via `loadComponent / loadChildren` keeps the initial bundle small and improves Core Web Vitals (especially with `@defer` blocks for view-level lazy loading of non-critical UI). The trade-off is discipline: OnPush bugs from in-place mutation are subtle, and migrating a legacy `NgModule` codebase to standalone incrementally requires careful handling of mixed-mode imports (standalone importing declared components requires keeping the owning `NgModule` around during transition).

Interviewer intent: An open-ended, judgment-based question testing whether the candidate can reason about trade-offs and real migration constraints, not just recite feature lists — this is where senior candidates differentiate from mid-level ones.

Q15. How does Angular's hierarchical injector system enable feature-level or per-instance service scoping, and give a concrete use case.

A: Because injectors are tree-shaped and any component can add `providers` that create a fresh element-injector-scoped instance for its subtree, you can scope a service's lifetime/state to exactly the part of the UI that needs it. Concrete example: a multi-step wizard component provides a `wizardStateService` in its own `providers` array; every instance of the wizard (e.g., opened twice in different tabs, or reused across navigations) gets an independent state object with no manual reset logic needed, while still using the same singleton pattern (`providedIn: 'root'`) for genuinely app-wide services like `AuthService` or `HttpClient`.

Interviewer intent: Checks whether the candidate can apply DI hierarchy knowledge to a real design decision (scoping state correctly) rather than only defining terms abstractly.

7. Quick Revision Cheat Sheet

- Angular = compiler (build-time, Ivy, `@angular/compiler-cli`) + runtime (`@angular/core`, browser).
- Building blocks: components, directives, pipes, services + DI, router, forms, HTTP client.
- Compiler emits `@ɵdefineComponent` with dual-mode (`rf & 1` creation, `rf & 2` update) instruction functions — no runtime HTML parsing.
- Ivy's "locality principle": a component's compiled output depends only on its own metadata/imports, not the whole module graph — this is what makes standalone components and fast incremental AOT possible.
- Standalone (default v17+, stable v14+): components declare their own `imports`; no NgModule required; bootstrap via `bootstrapApplication()` + provider functions (`provideRouter`, `provideHttpClient`, etc.) instead of `bootstrapModule()`.
- NgModule (classic): `declarations / imports / exports / providers / bootstrap; forRoot() / forChild()` avoid re-providing singletons in feature modules.
- DI hierarchy: element injectors (component/directive `providers`, DOM-tree-shaped) → environment injectors (`providedIn: 'root'`, module providers, lazy route boundaries) → platform injector. Modifiers: `@Self`, `@SkipSelf`, `@Host`, `@Optional`.
- `providedIn: 'root'` = tree-shakable singleton (factory attached to the class itself); NgModule `providers` arrays are not tree-shakable (module holds a live reference regardless of usage).
- Change detection default: zone.js patches async APIs → triggers `ApplicationRef.tick()` → full top-down tree walk re-running update instructions.
- `onPush`: subtree re-checked only on input reference change, internal/descendant event, async-pipe emission, or explicit `markForCheck()` / `detectChanges()`. Mutating inputs in place breaks this silently.
- Signals (`signal`, `computed`, `effect`) enable fine-grained reactivity and zoneless change detection (no zone.js, smaller bundle, precise re-renders).
- AOT (build-time) is the default/required production path; JIT (browser-time) is legacy. `strictTemplates` catches binding type errors at build time.
- View encapsulation defaults to `Emulated` (attribute-based CSS scoping, not real Shadow DOM) — global leakage and `::ng-deep` are common gotchas.
- Bootstrapping (standalone): platform injector → root environment injector (from `appConfig.providers`) → instantiate root component → insert into DOM → creation-mode render → app marked stable after async initializers resolve.

Created By - Durgesh Singh